CLASSIFICAÇÃO E PESQUISA DE DADOS

ÁRVORE PATRICIA

Aluna: Maria Júlia Marques Schettini
Prof: Samuel da Costa Alves Basílio

SUMÁRIO

- Árvore Trie
  - Aplicações
  - Funcionamento
  - Exemplo
  - Código
- Árvore Patrícia
  - Criação
  - Funcionamento
  - Exemplo
  - Comparação
  - Código

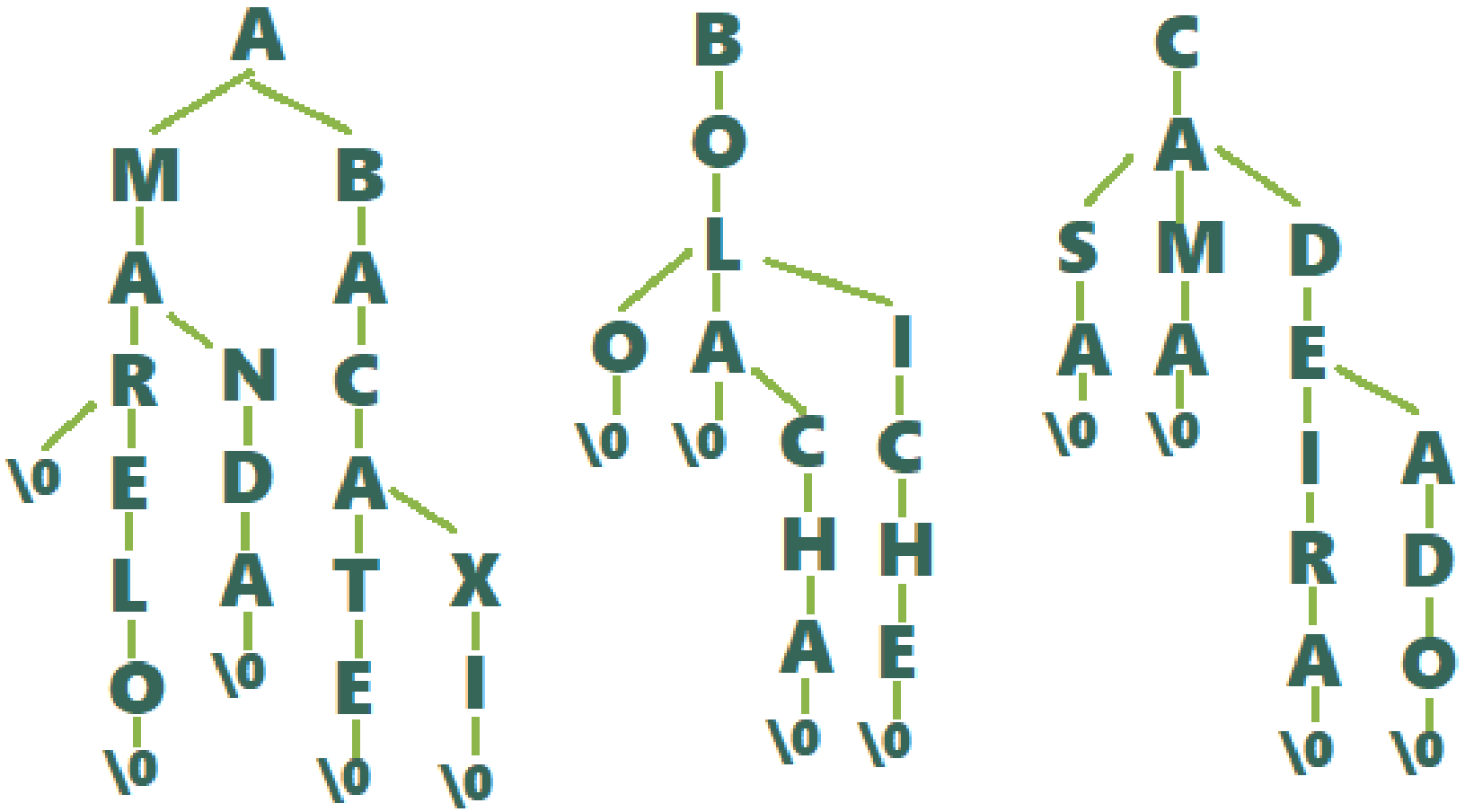
ÁRVORE TRIE

- Definida em 1960 por Edward Fredkin
- Vêm de Re**trie**val (Relacionado à Recuperação de Informações)
- Árvore N-ária
- Árvore de busca
- Utilizada comparando Strings
  - Dicionário
  - Preenchimento automático
  - Pesquisa em textos

ESTRUTURA DA ÁRVORE TRIE

- Chave é a palavra que deseja pesquisar na árvore
- Cada caminho possui um caractere
- Deve existir no nó uma condição de parada/término da palavra
  - Pode ser um boolean ou algum outro valor
- O percurso da raiz à condição de parada descreve a palavra
- Conforme desce um nível avança um elemento da chave

EXEMPLO


A
M
B
A
A
R
N
C
\0
E
D
A
L
A
T
X
O
\0
E
I
\0
\0
\0
B
O
L
O
A
I
\0
\0
C
C
H
H
A
E
\0
\0
C
A
S
M
D
A
A
E
\0
\0
I
A
R
D
A
O
\0
\0

CÓDIGO FONTE

```c
typedef int bool;
typedef bool TIPORET;
typedef struct no {
    struct no *filho[N_ALFABETO];
    TIPORET fim;
}no;
typedef no* pont;

pont criaNo (){
    pont p = NULL;

    p=(pont)malloc(sizeof(no));
    if (p){
        p->fim = false;
        int i;
        for (i=0;i<N_ALFABETO;i++)
            p->filho[i] = NULL;
    }
    return p;
}
int mapearIndice (char c){
    return ((int)c - (int)'a');
}
```

```c
void insere(pont raiz,char *chave){
    int nivel,i , tam = strlen(chave);
    pont p = raiz;

    for (nivel=0;nivel<tam;nivel++){
        i=mapearIndice(chave[nivel]);
        if(!p->filho[i])
            p->filho[i]=criaNo();
        p= p->filho[i];
    }
    p->fim = true;
}
bool busca (pont raiz, char *chave){
    int nivel, i,tam = strlen(chave);
    pont p = raiz;

    for (nivel=0;nivel<tam;nivel++){
        i=mapearIndice(chave[nivel]);
        if(!p->filho[i])
            return false;
        p= p->filho[i];
    }
    return (p->fim);
}
```

ÁRVORE PATRICIA

P ratical

A lgorithm

T o

R etrieve

I nformation

C oded

I n

A lphanumeric

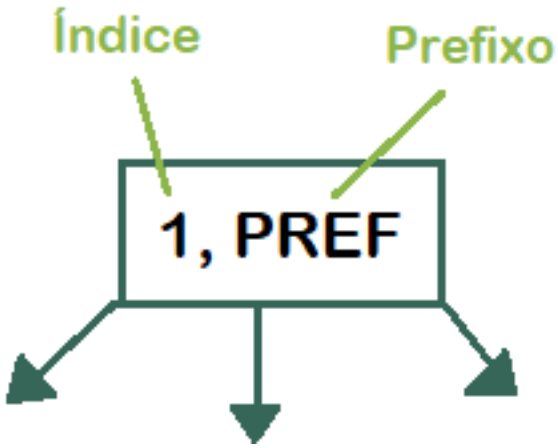
Algoritmo Prático para Recuperar Informações Codificadas em Alfanuméricas

ÁRVORE PATRICIA

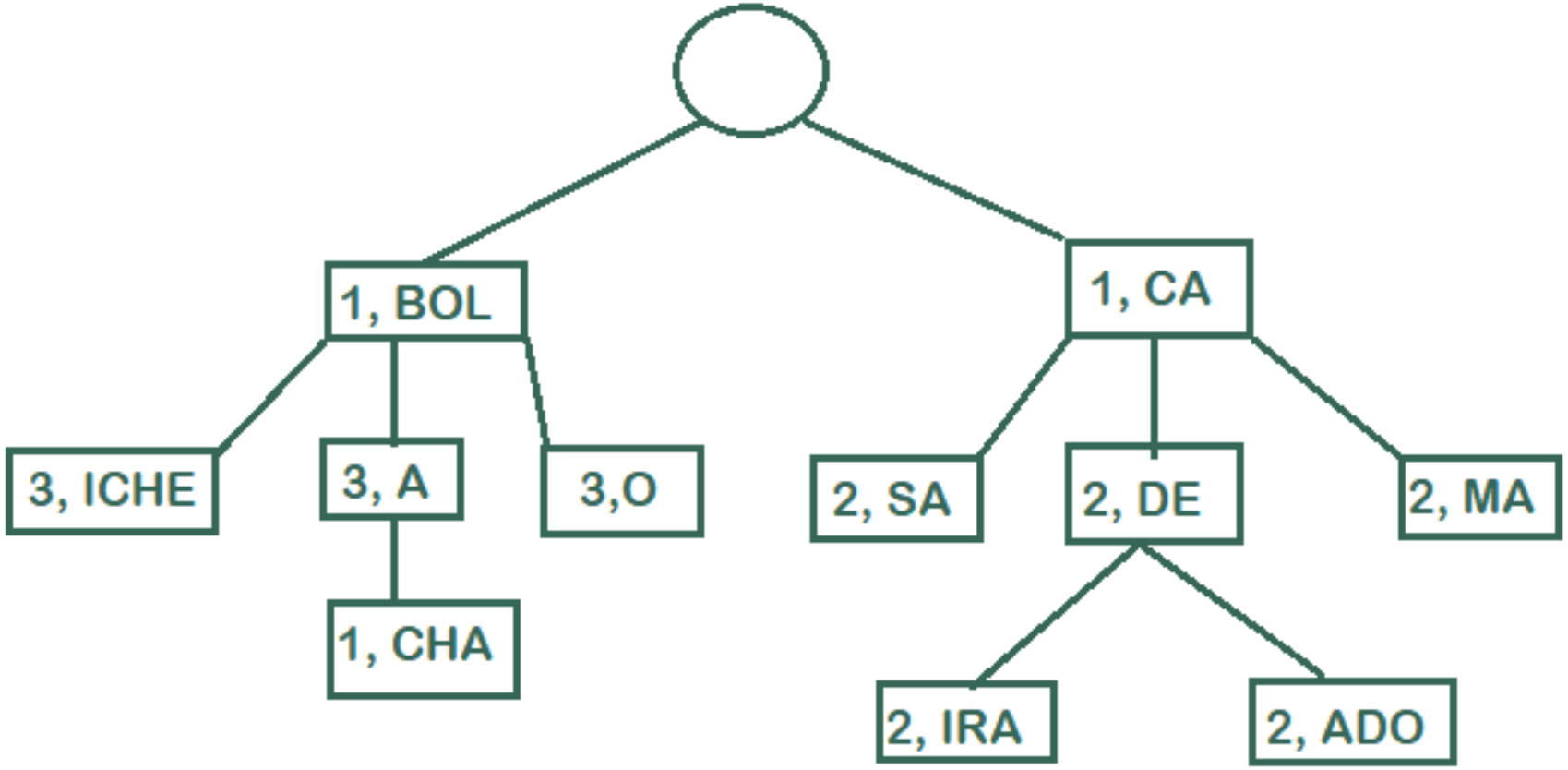
- Criada por Donald R. Morrison em 1968
- Trie Compactada
- Resolve o problema de caminhos de uma só direção
- Analisa através de um conjunto de caracteres e não letra a letra
- Palavra está contida no caminho da raiz até a folha ou até condição de parada (\0) ou até a folha
- Caminhos que possuem nós com apenas 1 filho são agrupados em uma única aresta

FUNCIONAMENTO

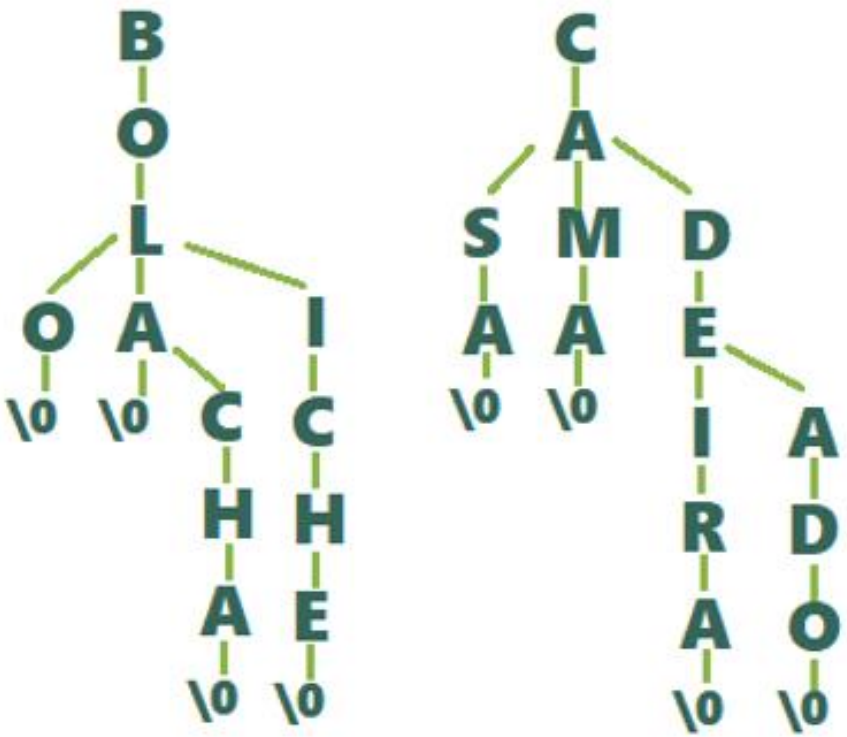
- Cada nó possui três campos: Índice (inteiro), Prefixo (String) e um vetor de nós
- Índice
  - Identifica qual a posição do caractere da chave que deve ser analisado
- Prefixo
  - Agrupa a parte em comum da palavra com a chave
- Filhos (vetor de nós)
  - O tamanho desse vetor depende do tamanho do alfabeto a ser utilizado
- Utilizando o índice e o prefixo consegue encurtar o caminho ao buscar/inserir palavras semelhantes

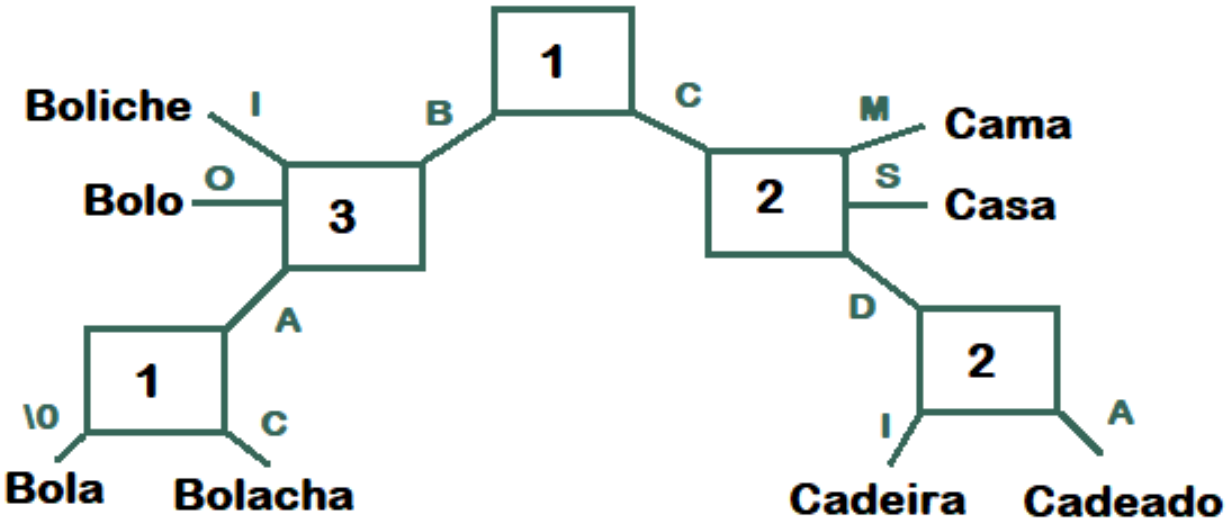


EXEMPLO



COMPARAÇÃO


B
O
L
O
A
I
\0
\0
C
C
H
H
A
E
\0
\0
C
A
S
M
D
A
A
E
\0
\0
I
A
R
D
A
O
\0
\0


1
Boliche
I
B
C
M
Cama
Bolo
O
3
2
S
Casa
A
D
1
2
\0
C
I
A
Bola
Bolacha
Cadeira
Cadeado

CÓDIGO

```c
#include<stdlib.h>
#include<stdio.h>
#include<string.h>
#define TAM 26

typedef struct NO{
    struct NO* filhos[TAM];
    int indice;
    char pref[20];
}NO;

typedef NO* Ramo;

void setFilhos(Ramo p)
{
    int i;
    for(i=0;i<TAM;i++)
        p->filhos[i] = NULL;
}

Ramo criaNo(){
    Ramo p = (Ramo)malloc(sizeof(
    if(p){
        setFilhos(p);
    }
    return p;
}
int mapearIndice(char c){
    return c - 'a';
}
```

```c
int isEmpty(Ramo raiz){
    if( raiz == NULL)
        return 1;
    return 0;
}

int verAlfabeto(char palavra[], NO raiz, int *indice){
    int i, j=0;

    for(i=0;i<strlen(palavra) && i<strlen(raiz.pref);i++){
        if(palavra[i] != raiz.pref[i]){
            j = 1;
            break;
        }
    }
    *indice = i;
    if(j){
        if(palavra[i] > raiz.pref[i])
            return 1;
        else
            return -1;
    }else{
        if(strlen(palavra)>strlen(raiz.pref))
            return 1;
        else if(strlen(palavra)<strlen(raiz.pref))
            return -1;
    }
    return 0;
}
```

CÓDIGO

```
void separaString(char *chave, int indice){
    char aux[20];
    int i, j;
    for(j=0, i=indice;i<strlen(chave);i++, j++)
        aux[j] = chave[i];
    aux[j]='\0';
    strcpy(chave,aux);
}
void insere(Ramo raiz, char chave[]){
    int i, v, tam_chave = strlen(chave);
    Ramo r = raiz;
    if(isEmpty(r->filhos[mapearIndice(chave[0])])){
        r = r->filhos[mapearIndice(chave[0])] = (NO*) malloc(sizeof(NO));
        setFilhos(r);
        strcpy(r->pref, chave);
        puts(r->pref);
        r->indice = tam_chave;
    }else{
        v = verAlfabeto(chave, *r->filhos[mapearIndice(chave[0])], &i);
        if(v==0){
            return;
        }else if(v==1){
            r->indice = i;
            r=r->filhos[mapearIndice(chave[0])];
            separaString(chave, i);
            insere(r, chave);
        }else{
            printf("\n\nInserir anteriormente não foi implementado!\n\n");
        }
    }
}
```

REFERÊNCIAS

- https://www.youtube.com/watch?v=4L6_5-92JGI&t=904s
- http://www.ufjf.br/jairo_souza/files/2009/12/6-Strings-Pesquisa-Digital.pdf
- https://marciobueno.com/arquivos/ensino/ed2/ED2_06_Trie_Patricia.pdf